Capítulo 6

Fase de Especificación de requerimientos

Objetivos del capítulo:

El objetivo de este capítulo es plantear el proceso de la fase de especificación de requerimientos. Por lo que se pretende:

- Mostrar todas las actividades para hacer la especificación de los requerimientos funcionales y no funcionales de un producto de software.(O1, O2)
- Aprender a definir las pruebas que demuestren que se cumplen los requerimientos del software especificados previamente. (O3)

6.1 Fase de Especificación de requerimientos: objetivos, actividades y productos.

El objetivo de esta fase es iniciar el desarrollo del producto de software. Por lo que se debe entender bien el problema a resolver para especificar los requerimientos del software y tener una descripción clara y no ambigua de lo que será el producto. Esta especificación debe proporcionar criterios para validar el producto terminado, los cuales se incluyen en el Plan de pruebas del software.

El proceso de desarrollo a emplear, a partir de esta fase, está basado en tecnología orientada a objetos y los modelos se construyen con la notación de *Unified Modeling Language* (UML) [Booch 99].

Objetivo:

O1. Entender el problema a resolver. (Ver 6.2)

Para construir un producto de software es necesario contar con la cooperación del cliente a fin de, entender cuál es el problema que desea resolver y cuáles son sus necesidades reales. El cliente usa términos de su entorno, cuyo significado no siempre es claro. Para evitar malos entendidos se establece un vocabulario común mediante un glosario de términos.

Actividades:

A1.1 Entender cuáles son las necesidades o requerimientos del software.

Producto:

- Definición del problema.
- Glosario de términos.

Objetivo:

O2. Proporcionar una descripción clara y no ambigua de las necesidades del software a través de la especificación de requerimientos funcionales y no funcionales. (Ver 6.3).

Para lograr la especificación de requerimientos se emplean varias técnicas, como la identificación de casos de uso, prototipo de la interfaz de usuario y la identificación de los requerimientos no funcionales.

Actividades:

A 2.1 Especificar los requerimientos funcionales con casos de uso.
A2.2 Construir el prototipo la interfaz de usuario.
A2.3 Identificar los requerimientos no funcionales.

Producto:

- La Especificación de requerimientos del software, que incluye:
  - Diagrama general de casos de uso, indicando el alcance en el primer ciclo.
  - Detalle de los casos de uso del primer ciclo.
  - Prototipo de la interfaz de usuario.
  - Lista de requerimientos no funcionales.

Objetivo:

O3. Obtener criterios para validar el producto de software terminado. (Ver 6.4).

Para saber si el producto que se desarrollará cumple con sus objetivos, hay que definir qué se espera de él, cómo se deberá comportar en situaciones normales y excepcionales.

Actividades:

A3.1 Construir el Plan de prueba del software.

Producto:

- Plan de prueba del software.

Formas a entregar en todas las fases:

- Semana del equipo
- Agenda y minuta de la reunión
- Lista de verificación
- Registro de defectos

Resumen de productos a entregar:

Producto	Responsable
Definición del problema	Administrador de desarrollo y Cliente
Glosario de términos	Administrador de desarrollo y Cliente
Especificación de requerimientos del software	Administrador de desarrollo
Plan de prueba del software	Administrador de desarrollo

6.2 Entender el problema

Para construir un producto de software, es necesario contar con la cooperación del cliente. Se debe entender cuál es el problema que tiene y cuáles son sus necesidades reales.

La mayor parte de las veces, el cliente no tiene claro qué es lo que realmente necesita. Es el desarrollador el responsable de ayudar al cliente a entender y expresar sus necesidades para que el software las pueda satisfacer. Para identificar los requerimientos se consultan a los interesados a través de varias técnicas [Tomayco] como son:

- Hacer entrevistas.
- Aplicar cuestionarios.
- Observar a los futuros usuarios al realizar las tareas que apoyará el software.
- Revisar documentos o sistemas ya existentes que se pretenden mejorar.

El cliente expresa de manera oral sus necesidades sobre el software que desea se le construya. Escribir un texto, entre el cliente y el desarrollador, que defina el problema, permite poner en blanco y negro las necesidades del software y ayuda al cliente a precisarlas. Por lo que la entrada inicial al proceso de desarrollo de software es escribir en un texto la *Definición del problema.*

El texto deberá escribirse preferentemente en un lenguaje que entienda el cliente sin términos técnicos computacionales.

Coad [Coad] propone una serie de estrategias al redactar ese texto, para ayudar a aclarar el objetivo del software. Una de ellas es escribir una frase del tipo "*para ayudar a"* o "*para apoyar a"* que permiten saber el tipo de usuarios que tendrá el software y para qué se usará. Otra estrategia que propone es que el desarrollador "*se dé una vuelta por el ambiente de trabajo donde se usará el software"*, esto permite ver a los usuarios y el tipo de cosas que necesitan para que el sistema los apoye en su trabajo diario. Una estrategia más, indica que "*se haga una lista de las características o cualidades"* que deberá tener el software, la que se puede escribir en orden de importancia.

Por ejemplo, en la figura 6.1 se muestra una definición del problema para construir un sistema para una bolsa de trabajo en internet. En él se aplican dos de las estrategias propuestas por Coad, usar la frase *"para apoyar a"* y *"hacer una lista de características*".

DEFINICION DEL PROBLEMA:

Bolsa de trabajo

Se desarrollará un sistema de bolsa de trabajo *que apoye a* desempleados y a empresas. Este sistema podrá ser usado por desempleados que están en busca de algún empleo y por empresas que estén en busca de empleados para ocupar una vacante.

Los desempleados podrán ver las vacantes disponibles sin tener que registrarse, pero si el desempleado quiere postularse para una vacante tendrá que registrarse forzosamente. Las empresas podrán publicar sus vacantes por medio de un contacto, el cual será el responsable de los datos de la empresa, la empresa podrá revisar quién se registró para ocupar la vacante publicada. La empresa podrá contactarlo después de revisar su solicitud y el currículo del desempleado.

Lista de características deseadas

1. Cualquier usuario podrá ver las vacantes disponibles en el sistema de software.
2. El usuario que se interese por alguna vacante en el sistema y quiera postularse a ella tendrá que registrar sus datos y agregar su currículo.
3. Las empresas tendrán que tener una clave de usuario y una contraseña para poder ver los postulados para su vacante.
4. Las empresas tendrán un representante, el cual será el encargado de la administración de los datos de la empresa y sus vacantes.
5. Las empresas sólo podrán ver a los desempleados que se registraron para sus vacantes.
6. El sistema estará creado en ambiente Web y podrá ser accedido mediante cualquier explorador Web.
7. El sistema deberá ser fácil de usar aún sin tener ningún tipo de capacitación para utilizarlo.

Figura 6. 1. Ejemplo de Definición del problema.

La definición del problema permite establecer un acuerdo y entendimiento común entre el equipo de desarrollo y el cliente sobre lo que se va a hacer. En esta definición se sugiere que participen todos los miembros del equipo de desarrollo para entender el propósito del software que van a desarrollar.

Glosario de términos

La definición del problema es difícil, pues el cliente usa los términos relacionados con el problema y el técnico el lenguaje de los desarrolladores. Para que se puedan comunicar más fácilmente todos los involucrados, se recomienda construir un *Glosario de términos* que establece un vocabulario común.

El Glosario de términos es un pequeño diccionario, donde se pone cada término y su significado para este proyecto. En este glosario se deben incluir también los términos familiares para el desarrollador con su significado. La construcción del glosario es un trabajo continuo a lo largo del proyecto, se inicia en esta fase y se concluye al finalizar el proyecto. En la figura 6.2 se muestra un extracto del glosario de términos del sistema de bolsa de trabajo.

GLOSARIO DE TÉRMINOS

Sistema de Bolsa de trabajo

Actor: Es una entidad que interacciona con el sistema para obtener un resultado. Puede ser una persona, otro sistema, un dispositivo, etc.
Bolsa de Trabajo: En este caso, es la concentración de información acerca de empresas que tienen puestos disponibles (vacantes) y de personas que tienen la necesidad de conseguir un empleo.
Caso de uso: Es la descripción de un conjunto de secuencias de acciones que un sistema lleva a cabo para regresar un resultado observable a un actor.
Contraseña: Clave de seguridad que se le asigna a un usuario.
Desempleado: Persona que no tiene trabajo y está en busca de un empleo.
Empresa: Organización que tiene la necesidad de contratar a personas con capacidades especificas para su correcto funcionamiento.
Nombre de usuario: Identificación del usuario para poder acceder al sistema.
Vacante: Puesto disponible para ser ocupado por un desempleado por parte de una empresa.
Visitante: Persona que abre las páginas del sistema y revisa la información que se presenta en este sitio Web, la información la podrá ver sin tener que registrar sus datos.

Figura 6.2. Glosario de términos.

6.3 Especificación de Requerimientos.

Los requerimientos son el punto de partida para el desarrollo de un producto de software. *Los Requerimientos de software son un área de la Ingeniería de Software dedicada al estudio de la adquisición, análisis, especificación, validación y administración de los requerimientos o necesidades de un software*. [SWEBOK 2001]. Es una fase fundamental

para el desarrollo de software con calidad. Se entiende por calidad en un producto de software aquel que cumple con las necesidades del usuario.

Un requerimiento o necesidad es lo que el cliente o un usuario necesitan que haga el software para resolver un cierto problema. Para definir los requerimientos deben colaborar conjuntamente varios roles: el Equipo de desarrollo, el Cliente (quien paga el software), los Usuarios (quienes lo usarán en su trabajo diario). Muchas veces el Cliente y el Usuario son la misma persona. A todas estas personas se les llama en inglés *stakeholders* que se traduce por *interesados* o *involucrados*.

La especificación de requerimientos es como un mapa para entender el problema a resolver. Los mapas son una ayuda para llegar más rápido al lugar deseado. Si no se tiene el mapa, se puede manejar a toda velocidad pero no necesariamente se llegará a la meta.

Una característica de los requerimientos es que cambian constantemente por muchas razones: se modifican las necesidades del cliente, cambia el ambiente, la tecnología, etc. por lo que, establecer los requerimientos es un proceso de negociación entre el cliente y los desarrolladores, donde ambos entienden y acuerdan el objetivo del software.

Los requerimientos deben formularse de forma clara, precisa y no ambigua. Para eso pueden usarse varias técnicas al mismo tiempo: el lenguaje natural, que es claro para el cliente pero ambiguo; el modelado gráfico, que es más claro para el desarrollador y no es ambiguo, pero puede no ser claro para el cliente; un prototipo de interfaz de usuario, útil para ambos pues es una representación visual de lo que hará el software.

Hay dos tipos de requerimientos: los funcionales y los no funcionales.

Los **funcionales** incluyen:

- Las entradas y salidas de datos, los cálculos o las funciones del software.

Los **no funcionales** son las características o restricciones que se esperan del sistema software:

- Necesidades de la interfaz externa como: tipo de usuario, hardware y software, comunicaciones, facilidades de uso requeridas por los usuarios.
- Atributos del software tales como: eficiencia, disponibilidad, seguridad, conversión, portabilidad, mantenimiento.
- Restricciones del diseño: de formatos, de archivos, lenguajes, estándares, compatibilidad.
- Otros: base de datos, instalación, etc.

El proceso para la **especificación de los requerimientos** es:

1. Entender las necesidades del software.
2. Identificar a los interesados en el sistema y solicitar los requerimientos.
3. Identificar y negociar los requerimientos funcionales y los no funcionales.
4. Hacer el prototipo de interfaz para que los interesados confirmen si esos son sus requerimientos.

5. Documentar la *Especificación de requerimientos*.

La especificación de requerimientos termina cuando se obtiene el documento de *Especificación de los requerimientos*, que resume lo que cliente necesita y que servirá de guía a los desarrolladores para analizar esos requerimientos, validarlos, administrarlos y generar el software. Validar un requerimiento implica comprobar que el software lo cumpla. Administrar los requerimientos significa que se lleve un control de los cambios que van surgiendo.

La Especificación de requerimientos es un documento donde se establece el acuerdo de lo que hará el software. Para hacer este documento se puede seguir lo que proponen los modelos de referencia, como MoProSoft [MoProSoft] para la estructura de este documento:

- Introducción. Es una descripción general del software y su propósito.
- Descripción de requerimientos. (Ver 6.5.2)

6.3.1 Requerimientos funcionales.

Una de las técnicas para especificar los requerimientos funcionales es la de los *casos de uso,* los que servirán de hilo conductor de todo el proceso de desarrollo. Esta técnica se utiliza para identificar los requerimientos funcionales y a partir de ellos se diseña, implementa y prueba el software. Permiten rastrear los requerimientos a través de todo el proceso de desarrollo hasta el producto terminado.

Diagramas de caso de uso

Los casos de uso proporcionan una manera incremental y modular de describir software. Definen como los usuarios utilizarán el software. El conjunto de casos de uso conforma el modelo de casos de uso que describe el comportamiento general del sistema. Los casos de uso proporcionan una representación que puede ser fácilmente comprendida por todos los interesados. Se representan gráficamente con *Diagramas de casos de uso* de UML.

Elementos de los diagramas de casos de uso

Caso de uso. *Un caso de uso es una descripción de un conjunto de secuencias de acciones que realiza el sistema para entregar a un actor un resultado o valor observable*. [Booch, 1999]. Se representa por una elipse con el nombre del caso de uso. El nombre del caso de uso debe iniciarse, preferentemente, con un verbo en infinitivo y se recomienda expresar funcionalidad en términos familiares al cliente y a los futuros usuarios. En la figura 6.3 se muestra un ejemplo de caso de uso del sistema de bolsa de trabajo, en el que se expresa que el sistema debe contar con alguna facilidad para consultar una vacante.

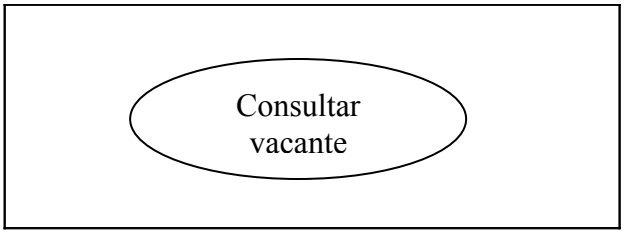


Figura 6.3. El caso de uso Consultar vacante para el sistema de bolsa de trabajo.

Actor. Es algo externo al software que intercambia información con el sistema, puede ser un usuario u otro sistema. El objetivo de un actor es completar una funcionalidad con el software para obtener un valor o servicio. Se representa con una figura humana con el nombre del actor en singular. Los sistemas externos con los que interacciona el software que se está desarrollando también son actores. Un caso de uso puede proporcionar un valor a uno o más actores. En el ejemplo del sistema de bolsa de trabajo (figura 6.4), un actor es la Empresa, quien representa a todas las posibles empresas que pueden administrar los datos de sus vacantes en el sistema o consultar los datos de los desempleados que se registraron para dichas vacantes.

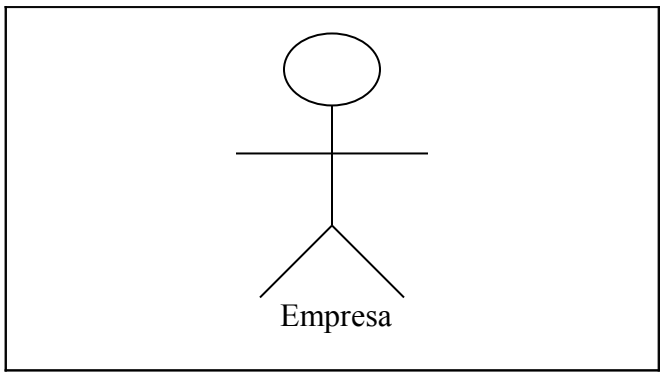


Figura 6.4. Un actor para el sistema de bolsa de trabajo.

Alcance del sistema. Representa la frontera del sistema y contiene los casos de uso que se realizan en cada ciclo de desarrollo. Se representa por un rectángulo que incluye los casos de uso que están dentro del alcance.

Diagrama de casos de uso. Un diagrama de casos de uso incluye los actores identificados, los casos de uso para cada actor y el alcance del sistema En el diagrama general de casos de uso se colocan las funcionalidades o casos de uso más generales, que posteriormente se detallan en diagramas en los que se desglosa cada funcionalidad. Este diagrama general tiene por objetivo mostrar de forma gráfica y clara las funcionalidades del sistema por lo que deberá ser simple, para mostrar a golpe de vista el alcance del sistema. Se recomienda que siempre se incluya un caso de uso para entrar al sistema y uno de salir para los actores. El diagrama general se discute con el cliente y los usuarios del software.

En la figura 6.5 se muestra el diagrama general de casos de uso del sistema de bolsa de trabajo. En este diagrama se aprecian 8 casos de uso generales que permiten entender la funcionalidad general del sistema y su alcance en el primer ciclo. Posteriormente cada uno de los casos de uso se detallará.

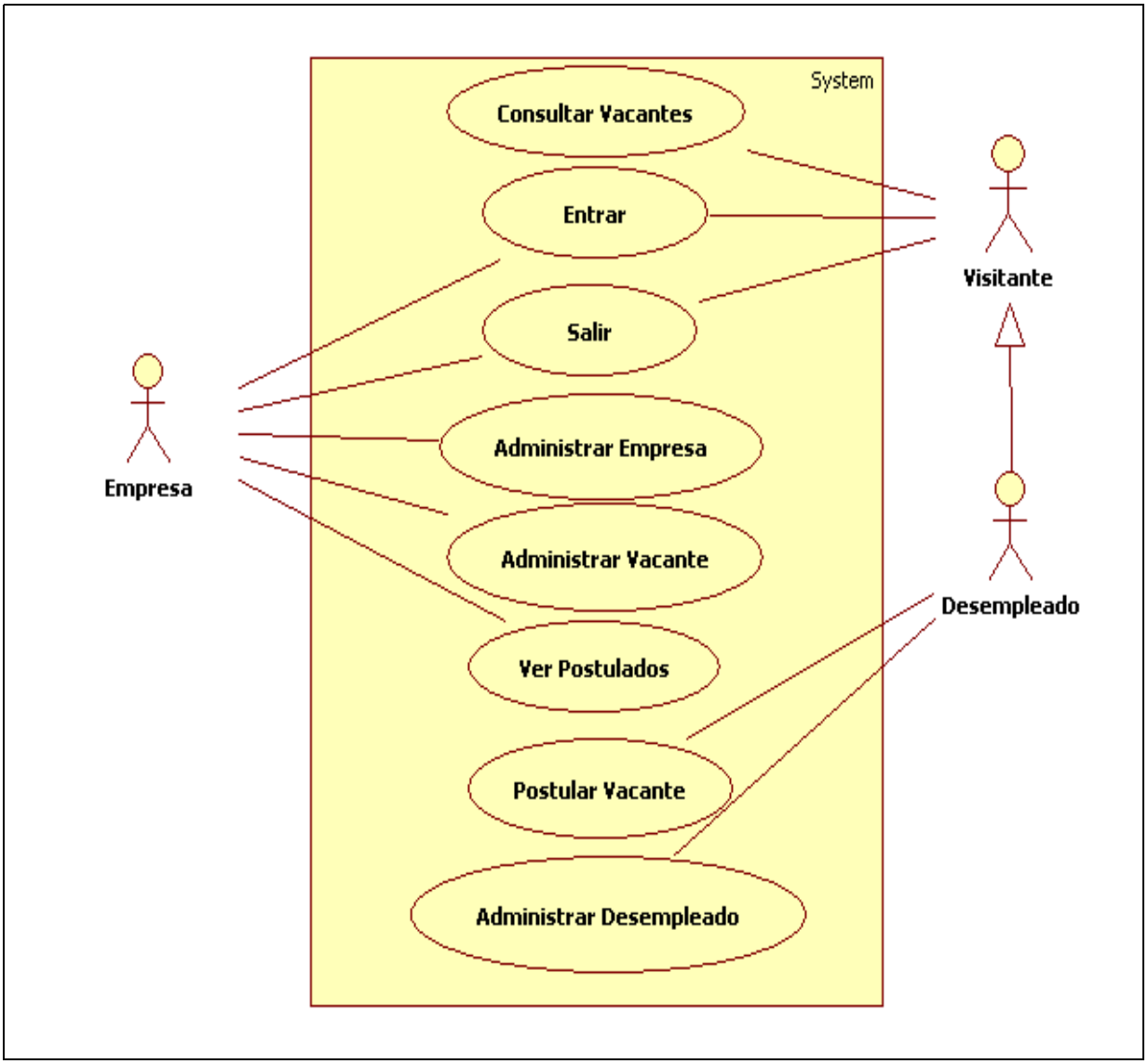


Figura 6.5. Diagrama general de casos de uso del sistema de bolsa de trabajo.

Proceso para la **creación de diagramas de casos de uso**:

1. Identificar los actores del sistema. Esto es, los que interaccionarán con el software para obtener un resultado de él.
2. Identificar las funcionalidades generales para cada actor y representarlos como casos de uso.
3. Definir el alcance o los casos de uso que se desarrollarán en el ciclo.
4. Detallar cada caso de uso.

Criterios para identificar a los actores:

1. Para identificar a un actor se revisa la definición del problema a fin de definir el tipo de usuarios u otros sistemas que interactuarán con el software. Se nombran con mayúscula y en singular.
2. Los candidatos para actores son roles generales no personas concretas. Por ejemplo Solicitante y no Juan Pérez.
3. Los casos de uso representan las necesidades de los usuarios del sistema que podrán ser modeladas y validadas en los casos de uso.
4. Para identificar los casos de uso, se revisa la definición del problema para cada actor, buscando las funcionalidades generales que requiere.

Detalle de los casos de uso

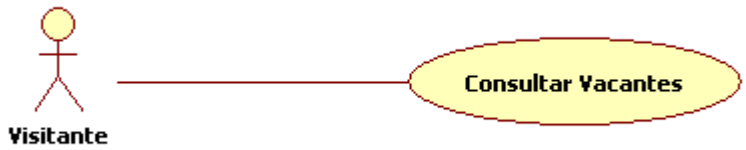
Una vez identificados los principales casos de uso para cada actor, se describe en detalle cada uno. La plantilla que se propone para esta descripción es:

> **Nombre**: El nombre deberá ser un verbo en infinitivo representativo de la funcionalidad del caso de uso.
> **Actor:** Nombre en singular del actor encargado de iniciar la acción y que recibe el resultado del caso de uso.
> **Diagrama detallado** del caso de uso indicando el actor y sus variantes o detalles.
> **Descripción:** Texto breve describiendo la acción.
> **Precondiciones**: Acciones previas del estado del sistema para que se pueda iniciar el caso de uso.
> **Flujo de eventos normales**: Tabla que describe el flujo de acciones entre el actor y el sistema durante el caso de uso.
> **Flujo de eventos alternativos o excepcionales**: Tabla con las acciones que ocurren en situaciones anormales o excepcionales.
> **Poscondiciones**: Define el estado del sistema después de la terminación exitosa del caso de uso.

En la figura 6.6 se muestra el detalle del caso de uso Consultar vacante del sistema de bolsa de trabajo.

CASO DE USO: CONSULTAR VACANTE

Actor: **Visitante**



Descripción: Un **Visitante** entra al sistema para consultar las vacantes que están disponibles.

Precondiciones:

- El visitante conoce la dirección Web del sistema de software SIBOT (nombre del sistema de bolsa de trabajo).
- El visitante desea ver las opciones de trabajo que están disponibles.

Flujo de eventos normales:

Usuario		Sistema		
Paso	**Acción**	**Paso**	**Acción**	**Excepción**
1	El usuario selecciona el vínculo Ver Vacantes.	2	Muestra la pantalla de Todas las Vacantes.	E1
3	Selecciona la Vacante de la cual desea ver los datos a más detalle.			
4	Oprime el botón Ver	5	Se muestra la pantalla de Detalle de Vacante con los datos de la vacante seleccionada.	E1,E2

Flujo de eventos alternativos:

Id	Nombre	Acción
E1	La conexión con la base de datos no está establecida o se interrumpió.	Se manda un mensaje de error, el cual indica que los datos no se pueden mostrar debido a que no hay conexión con la base de datos.
E2	No se ha seleccionado ninguna vacante	No hace nada.

Poscondiciones:

- Los datos de la vacante se muestran a detalle.

Figura 6.6. Detalle del caso de uso Consultar Vacante del sistema de bolsa de trabajo.

6.3.2 Prototipo de interfaz de usuario

Un prototipo de interfaz de usuario es una representación parcial de la interfaz de usuario que tendrá el software, que muestra la forma en que el usuario interaccionará con él. Este prototipo es un elemento muy importante para la comunicación con el usuario en la definición de los requerimientos, pues al revisar la interfaz, el usuario puede refinar sus necesidades y comunicarlas al desarrollador.

Se llama *pantalla* o interfaz al mecanismo con el cual interactúa el usuario con el sistema. Cuando se diseñen estas *pantallas* podrán ser código html, o ventanas o entradas de modo texto.

Para diseñar el prototipo de la interfaz se consideran los casos de uso planteados en el diagrama general y se puede construir al mismo tiempo que se detallan los casos de uso. Si el diagrama general tiene los casos de uso X, Y y Z, en la pantalla principal del sistema deberán estar las opciones del menú X, Y y Z. Si en la descripción del flujo de un caso de uso dice "el sistema presenta la pantalla de X...", en el prototipo deberá existir esa pantalla. Si en el flujo dice "el usuario oprime el botón M...", deberá haber un botón M en la pantalla.

En resumen, se debe cuidar que exista coincidencia entre el detalle de los casos de uso y el prototipo. Deben coincidir:

- Las opciones del menú y los casos de uso
- Los nombres de las pantallas
- Los nombres de los botones

En la figura 6.7 se muestra un ejemplo del prototipo de la interfaz del detalle de caso de uso **Alta de Vacante**.

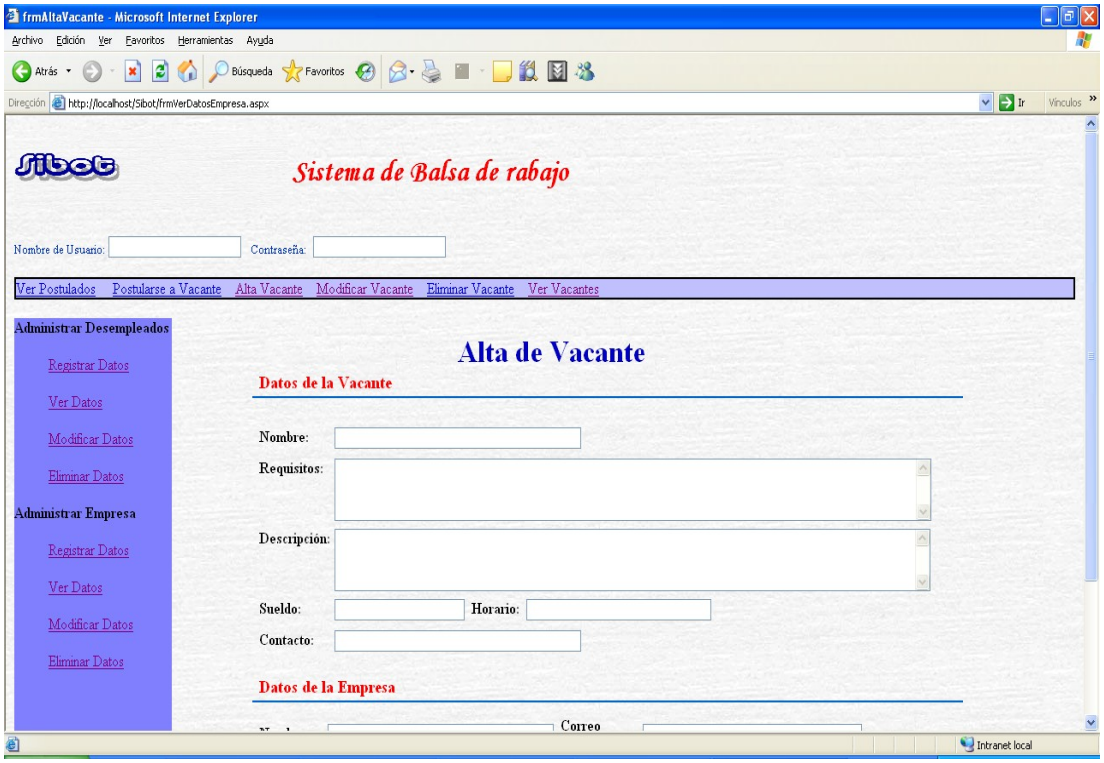


Figura 6.7. Pantalla para el caso de uso Alta de Vacante.

6.3.3 Requerimientos no funcionales

Los requerimientos no funcionales tienen que ver con los atributos o características que el cliente solicita para el funcionamiento del software.

Los requerimientos no funcionales pueden referirse a:

- **Interfaz con el usuario:** descripción de las características que permitan al software apoyar al usuario en sus tareas.
- **Interfaz externa**: relación o conexión con otros sistemas.
- **Confiabilidad**: solicitud del desempeño respecto a seguridad, tolerancia a fallas y su recuperación.
- **Eficiencia:** límites de tiempo de respuesta.
- **Mantenimiento**: facilidad de comprensión y realización de modificaciones futuras.
- **Portabilidad**: facilidad de transferencia de un ambiente de ejecución a otro.
- **Restricciones de diseño y construcción:** las que imponga el cliente.
- **Legales y reglamentarios:** necesidades impuestas por leyes o reglamentos de otros.

Los requerimientos no funcionales especificados para el sistema de la bolsa de trabajo se muestran en la figura 6.8.

REQUERIMIENTOS NO FUNCIONALES

Interfaz con usuarios

- El sistema debe de ser fácil de usar, además deberá tener una interfaz gráfica agradable y con colores suaves para no dañar en algún sentido visual al usuario.
- El funcionamiento del sistema deberá estar activo las 24 horas del día y los 365 días del año.

Confiabilidad

- Este sistema deberá de tener una alta confiabilidad e integridad con los datos de los usuarios.
- Para poder modificar, dar de alta o eliminar datos, el usuario debe de proporcionar al sistema un nombre de usuario y una contraseña.

Eficiencia

- La velocidad para mostrarle los datos al usuario debe de ser considerable, lo cual implica que la pagina no debe de contener imágenes que haga que el sistema se retarde.

Restricciones de diseño y construcción

- El sistema de software será construido para funcionar en ambiente Web.
- El sistema deberá estar codificado en lenguaje de programación Java.
- La base de datos del sistema de software deberá estar construida con el manejador de bases de datos MySQL.
- Para el desarrollo de este sistema se utilizarán las siguientes herramientas
    - ArgoUML para modelado del sistema.
    - Java para la codificación.
    - Microsoft Word para hacer la documentación.
    - Tomcat para el desarrollo del sitio web.
    - MySQL para la creación de la Base de Datos.

Figura 6.8. Requerimientos no funcionales para el sistema de bolsa de trabajo.

6.4 Plan de prueba del software

El objetivo del *Plan de prueba del software* es identificar las pruebas que son necesarias para asegurar el cumplimiento de los requerimientos especificados. Uno de los objetivos de especificar los requerimientos es poder rastrearlos a lo largo del desarrollo y validar que el producto que se construya cumple con ellos. Se mencionó que los casos de uso facilitan este rastreo, por lo tanto en el Plan de prueba del software se especifican casos de prueba para todos los casos de uso en situación normal y excepcional. De esta manera se tendrá la seguridad que el software cumple con las necesidades del cliente y no falta ni sobra ninguna funcionalidad.

La definición del Plan de prueba del software se puede realizar a la par del detalle de los casos de uso. Este plan consiste en una lista de casos de prueba para el flujo normal y para los flujos excepcionales de cada caso de uso. Al diseñar los casos de prueba se pueden hacer ajustes al detalle de los casos de uso pues se entienden mejor los requerimientos.

Los casos de prueba para cada caso de uso se documentan en una tabla. Un caso de prueba identifica las entradas al caso de uso y los resultados esperados. Se consideran entradas para flujos de eventos normales y eventos alternativos o excepcionales. En la figura 6.9 se muestran los casos de prueba que se determinaron de acuerdo a los flujos del caso de uso Consultar vacante del sistema de bolsa de trabajo.

CASO DE USO: CONSULTAR VACANTE

Caso de Prueba 1.	
Entradas	**Resultado esperado**
Selección de la opción Ver Vacantes, por parte del usuario. Selección del botón Ver de la vacante seleccionada por parte del usuario.	Pantalla Todas las Vacantes. Detalle de la vacante seleccionada
Caso de Prueba 2.	
Entradas	**Resultado esperado**
Selección de la opción Ver Vacantes, por parte del usuario.	Mensaje de error que indica que los datos no se pueden mostrar debido a que no hay conexión con la base de datos.

Figura 6.9. Casos de prueba para el caso de uso Consultar vacante del sistema de bolsa de trabajo.

6.5 Profundización de los temas para el segundo ciclo

6.5.1 Captura de requerimientos ágiles

Los métodos ágiles son los procesos de desarrollo de software que se consideran menos *burocráticos* en cuanto a la cantidad de documentación que se debe generar y mas orientados a las personas. Otra característica importante de estos métodos es que son iterativos e incrementales de igual manera como son TSPi [Humphrey] y el Proceso Unificado [Jacobson]. La diferencia fundamental entre estos dos métodos y los llamados ágiles, es que consideran iteraciones más cortas, de 3 o 4 semanas en las que se entregan productos parcialmente ejecutables al cliente. Ejemplos de estos métodos son XP, SCRUM y otros [Larman].

Los procesos de desarrollo de métodos ágiles tienen como algunos de sus principios [Manifiesto Ágil] los siguientes:

- La satisfacción del cliente
- Considerar el constante cambio de los requerimientos
- Entregar un producto funcional con frecuencia
- El trabajo conjunto con los clientes
- La forma mejor de obtener los requerimientos es con reuniones frecuentes cara a cara con los clientes.

Es una interpretación errónea de estos principios que no deban escribirse ni modelarse los requerimientos. En las primeras iteraciones, el principal producto que se obtiene es la clarificación de los requerimientos (hasta 80% en las primeras 4 iteraciones) y un 10% en la construcción del producto. [Larman]

Para la especificación de los requerimientos se propone no generar documentos tan formales como los aquí mencionados. Se proponen varias técnicas diferentes a las establecidas en este capítulo, pero todas llevan a definir con claridad las necesidades del software.

Algunos métodos usan los casos de uso detallados en forma de escenarios que se implementan contemplando los aspectos de la arquitectura, las interfaces de usuario hasta el manejo de la base de datos. [Cockburn]

Las *historias de usuario* las propone XP como método para capturar y priorizar los requerimientos. En una tarjeta se escribe la necesidad de usuario y se asigna a un programador quien lo comenta con el cliente hasta que está listo para generar el código necesario.

Otra técnica es hacer el prototipo de la interfaz de usuario en papel y representar las pantallas o páginas web en un pizarrón en el que se pega una representación de la interfaz e indicando la navegación entre ellas. Esta técnica se llama *GUIs con pegamento* que permite capturar los requerimientos y la visión de cómo se interactúa en el sistema.

Para otras técnicas se puede consultar [Larman].

Como se puede apreciar, las técnicas y modelados expuestos en este capítulo son útiles en cualquier enfoque de procesos de desarrollo que se adopte ya sea ágil o disciplinado.

6.5.2 Estándar de especificación de requerimientos de MoProSoft

Según MoProSoft la especificación de requerimientos se compone de una introducción y una descripción de requerimientos:

Introducción: Descripción general del software y su uso en el ámbito de la organización
Descripción de requerimientos:

- Funcionales: necesidades que debe satisfacer el software.
- Interfaz de usuario: definición de las características de la interfaz de usuario incluyendo la descripción del prototipo.
- Interfaces con otro software o hardware
- Confiabilidad: nivel de desempeño con respecto a la tolerancia a fallas y recuperación.
- Eficiencia: desempeño del software con respecto al tiempo y utilización de recursos.
- Mantenimiento: elementos que facilitarán las modificaciones del software.
- Portabilidad: características que permitan su transferencia a otros ambientes.

- Restricciones del diseño y construcción: necesidades impuestas por el cliente.
- Legales y reglamentarios: necesidades impuestas por leyes y reglamentos.

Prácticas de esta fase

Práctica 6 Diagramas de caso de uso.
Práctica 7 Prototipo de interfaz de usuario.
Práctica 8 Requerimientos no funcionales y Plan de prueba del sistema.

Referencias bibliográficas

- Booch G., J. Rumbaugh, I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley. 1999
- Coad P. at. al. *Object Models, Strategies, Patterns and Applications*. Yourdon Press Computing Series, Prentice Hall. 1995
- Cockburn A. *Agil Software Development*. Addison Wesley 2002.
- Endres A., Rombach D. *A Handbook of Software and Systems Engineering.Empirical Observations, Laws and Theories*. Pearson/Addison Wesley 2003.
- Humphrey, W. *Introduction to Team Software Process*. SEI Series in Software Engineering Addison Wesley. 2000
- IEEE estándar 830 de 1984.
- Jacobson I., Booch G., J. Rumbaugh. *The Unified Software Development Process*. Addison-Wesley. 1999
- Larman C. *Agile and Iterative Development. A manager's Guide*. Addison Wesley 2004.
- MoProSoft Modelo de Procesos para la Industria de Software. Versión 1.3 Agosto 2005. www.software.com.mx
- Pressman Roger S., "Ingeniería del software. Un enfoque práctico". McGraw Hill 3ª Ed. 1993.
- SWEBOK. *Guide to the Software Engineering Body of Knowlwdge*. Trial version Mayo 2001. www.swebok.org
- Tomayko J. E., Hazzan O. Human Aspects of Software Engineering. Charles River Media, Computer Engineering Series. 2004.